FernUniversität in Hagen
Faculty of Mathematics and Computer Science
Artificial Intelligence Group

Seminar: Artificial Intelligence - Visual Computing
3D Gaussian Splatting

Seminar Paper
submitted by
Sebastian Frehmel (version: 2026-08-04 14:59:03)

Abstract

3D Gaussian Splatting (3DGS) represents a seminal approach to 3D reconstruction and novel-view synthesis first published in 2023 by Kerbl et al. This paper introduces the underlying historical developments, motivation, and concepts of 3DGS for a Data Science audience. We examine the 3DGS algorithm in full detail with an emphasis on dissecting the constituent steps of the training procedure.

1. Introduction to the Field of 3D Reconstruction

3D Gaussian Splatting is a new technology in the field of 3D reconstruction which requires introducing certain basic terminology.

3D reconstruction marks a field in Computer Graphics which allows creating 3D *scenes* (e.g. individual objects, indoor environments, or large-scale outdoor spaces) from a given input like an overlapping set of images. Once this 3D scene has been created, *novel-view synthesis* refers to the process of gaining new 2D imagery from this 3D scene [16, 28] using camera positions not found in the initial imagery. However, both processes are commonly aggregated into the single term 3D reconstruction, a practice we do not adopt here. Next, *real-time novel-view synthesis* is the ability to synthesize new 2D imagery at a rate of 24 to 30 frames per second (FPS) or more [7], making it possible to smoothly explore a reconstructed 3D scene comparable to e.g. 3D video games.

First efforts in this field date back to the 1990s with work on multi-view geometry [16] and remained a niche area of research due to lack of powerful models until the general emergence of neural networks in the late 2010s. Neural networks for the first time allowed creating 3D scenes with realistic image quality using the radiance field approach. An *implicit radiance field* [9] is based on the mathematical concept of the five-dimensional plenoptic function $L : x, y, z, \delta, \phi \rightarrow c, \alpha$ which expresses a scene by continuously mapping any given point (x, y, z) in space and a set of viewing angles δ and ϕ (pitch and yaw) to a color c and an opacity α at that point. *Explicit radiance fields* restrict the plenoptic

function to a finite set of points, for example using a rigid 3D *voxel* grid.

One well-known implementation of radiance fields are *Neural Radiance Fields* (NeRFs) introduced by Mildenhall et al. in 2021 [20]. NeRFs take a set of input images and use a neural network to learn an implicit radiance field representation of the resulting scene, which means the full scene is encoded in the neural network's weights. In order to perform novel-view synthesis for a NeRF, a technique called *volumetric ray marching* is employed. For every pixel in the novel view, the NeRF algorithm transmits a ray through the implicit radiance field resulting in a very high number of queries to the neural network for each (discretized) point along the path of the ray before adding up all predicted RGB and opacity values to form the final color value of each pixel [20].

The other aspect which is important to know is the concept of *splatting*. First introduced in 1991 by Westover [33] and refined later in the early 2000s by a group around Zwicker [40, 41], it takes the reverse approach to ray marching by projecting all elements in the 3D scene onto the view plane towards the camera, a process which colloquially can be called "splatting" or "squashing". It was also Zwicker et al. in the same publications who introduced *3D Gaussians* as a data structure to simplify and extend splatting.

Lastly, there are important basic concepts in Computer Graphics besides commonly known information like the *RGB* color model, that need explaining especially in the context of splatting, starting with *world space*, *view space*, and *image space*. The first describing the space in which the 3D scene is built; the second describing the same space shifted with the camera at the origin; and the third describing the space after *camera projection*. Then there is the near plane, also called the *view plane* or projection plane and the *far plane* as well as the *camera pose* or position. The camera pose and its *field of view* (FOV) decide which elements of the scene between the view plane and the far plane are to be included in the projection onto the view plane. This setup is known as the "pyramid of vision" or *view frustum* as it takes the shape of a frustum, see Figure 1 on page 13. *Frustum culling* removes all elements outside the view frustum. For more general aspects of computer graphics see [17].

2. Motivation for 3D Gaussian Splatting

Training NeRFs to reach photorealistic scene quality can take several days on professional high-end GPU hardware, as is the case with Mip-NeRF [3], a NeRF-based system reaching highest Peak Signal-to-Noise Ratio (PSNR) scores, used to evaluate quality of reconstruction as perceived by a human [12, 3]. Also, during novel-view synthesis, ray marching is very time-consuming as the neural network has to be queried very often and the algorithm cannot determine if a ray has reached sufficient opacity [12]. In other words, a ray always has to travel through the entire implicit radiance field representation of the neural network. This has led to some implementations of NeRFs switching to explicit radiance field representations and losing the neural network for novel-view synthesis altogether (e.g. "Plenoxels" [6]), which comes at the inherent cost of reduced image quality due to fewer and less precise points in the radiance field. These approaches do allow training times of 20–30 minutes and make it possible to come close to real-time

novel-view synthesis reaching around 10–15 FPS at 1080p resolution. Other downsides of NeRFs include possibly having to sample empty space along the ray, as well as not being able to easily edit or amend a reconstructed scene as these details are all hidden in the weights of the neural network ("black box"). Therefore, either NeRFs allow state-of-the-art reconstruction quality at the cost of long training and synthesis times while only being able to synthesize single images, or they have to reduce reconstruction quality to be able to reduce training times and come somewhat close to real-time novel-view synthesis [12].

To overcome the mentioned drawbacks of NeRFs, Kerbl et al. [12] at INRIA in France introduced 3D Gaussian Splatting (3DGS) in late 2023, which makes it a very recent technology. Yet, their paper has had a fundamental impact in the field of 3D reconstruction and beyond, accumulating nearly 10,000 citations [26, 24] in three years.

Kerbl et al. managed to combine previously existing concepts of 3D Gaussian Splatting and Differential Rendering among others to create a learnable explicit radiance field representation. Key to this approach is the fact that they used a decomposition of the covariance matrix of 3D Gaussians that makes them differentiable and eligible for standard backpropagation procedures using gradient descent. Most importantly, this means that 3DGS does not use neural networks.

A 3D Gaussian can best be imagined as an ellipsoid "fuzzy blob" that loses opacity towards its edges while being able to take many shapes, from a perfect sphere to cigar-like shapes, elongated needles, or flat, round shapes. Using them as the base primitive for 3D reconstruction allows for arbitrary precision of their position, bypassing issues explicit radiance field representations had when used in NeRFs. With 3D Gaussians also comes direct editability of the scene, such as additions or lighting changes, making it much more accessible and subsequently usable for downstream applications.

The other important cornerstone in the publication by Kerbl et al. was the major engineering achievement of mapping this new idea perfectly onto modern GPU hardware for both 3D reconstruction and novel-view synthesis, resulting in speed-ups that had not been seen before in this field. The centerpiece here is the tile-based rasterizer, which we will cover in-depth in Chapter 4.2.

This way they created the first explicit, differentiable, unstructured, and primitive-based radiance field architecture finally allowing real-time novel-view synthesis at 1080p resolution while matching or even improving scene quality in comparison to state-of-the-art NeRF architectures [12].

The original paper by Kerbl et al. appears very dense, leaving out many details expecting an expert audience, or moving them to a project website [10] and a GitHub repository [11]. Even most subsequent work does not discuss the algorithmic approaches, which have seen only minor alterations since their introduction. Therefore, this seminar paper takes a deep dive into the algorithm for an audience outside of Computer Graphics (currently not taught at Fernuniversität Hagen), while emphasizing machine learning aspects. From now on, "the algorithm" denotes the 3DGS algorithm used for 3D reconstruction at all stages.

We will begin with 3D Gaussians as the fundamental data structure and continue by dissecting the training procedure following the original Kerbl et al. paper and GitHub closely.

3. 3D Gaussian Data Structure

An analysis of the original authors' [12] repository [11] or consulting survey papers like [2] presents a 3D Gaussian as a data structure consisting of 14 to 59 attributes:

- Position μ: three-dimensional coordinates in world space for the origin of the 3D Gaussian. (3 attributes)
- Opacity o: the peak or base opacity at point μ. It can be any number but is passed through a sigmoid function to bound it to $(0, 1)$. (1 attribute)
- Covariance matrix Σ: A 3×3 matrix determining 3D shape and rotation of the 3D Gaussian, but decomposed into three scaling values and four rotational values called a *quaternion*. Decomposing the covariance matrix is pivotal for the backpropagation process, see Chapter 3.1. (7 attributes)
- *Spherical Harmonics* (SH) coefficients c: model view-dependent colors and specular reflections. Spherical Harmonics come in four degrees [36]. SH0 is a simple RGB value denoting a solid color of the 3D Gaussian that is not dependent on viewing angles. Degrees SH1 to SH3 contribute 3, 5, and 7 attributes per color channel each, increasing the level of realism. SH3 is recommended for 3DGS [12]. Spherical Harmonics and their complexity are out of scope for this paper, and we will implicitly mean SH when referring to any notion of color. (3-48 attributes)

The reason a 3D Gaussian is called *Gaussian* lies in its opacity distribution α which is modelled after the dimension-agnostic form of the multivariate normal distribution [41]:

$$\alpha(p) = o \cdot \exp\left(-\frac{1}{2}(p-\mu)^T \Sigma^{-1}(p-\mu)\right)$$

with p being any point in world space and Σ^{-1} the inverse of the covariance matrix. In other words, we multiply the peak opacity value with a probability given by the three-dimensional normal distribution shaped by Σ. This way, a 3D Gaussian becomes rapidly more transparent toward its edges. Mathematically, each 3D Gaussian reaches into infinity with values asymptotically near 0 the farther from the center the opacity is being calculated. To remedy this effect, 3DGS enforces a hard 3-Sigma boundary meaning that everything outside three standard deviations from μ is culled, establishing an ellipsoid bounding shape which captures 99.7% of the 3D Gaussian.

The shape a 3D Gaussian takes is therefore only defined by the density of its opacity distribution or as it is usually visualized, by its ellipsoid bounding shape.

3.1. 3D Gaussian Covariance Matrix Decomposition

For the entire learning process to work with backpropagation and gradient descent, a 3D Gaussian has to be differentiable. The original inception of a 3D Gaussian by Zwicker et al. [41] did not account for differentiability (or spherical harmonics for that matter), they were focussed on elaborating on the concept of splatting for existing scenes.

During backpropagation, the algorithm would have to determine new, better values for

the covariance matrix Σ toward the direction gradient descent is pointing. However, in order to make sense geometrically, a covariance matrix in this case has to be positive semi-definite which means the eigenvalues have to be $>= 0$. As running a loop during backpropagation to find new valid values for the entire covariance matrix until the whole matrix is positive semi-definite again while still satisfying gradient descent would be far too costly, Kerbl et al. [12] used a clever decomposition of the covariance matrix first proposed in 2019 [35] or later by a group involving team members of the Kerbl paper [14]:

Σ can be decomposed into a 3×3 scaling matrix S and a 3×3 rotational matrix R: $\Sigma = RSS^TR^T$ which is itself just an instance of the default eigendecomposition $\Sigma = Q\Delta Q^T$ with eigenvectors Q (rotations) and a diagonal matrix of eigenvalues Δ (scaling). Lastly, since S is a diagonal matrix, $SS^T = S^2 = \Delta$.

The scaling matrix can now be decomposed into three scaling values (s_1, s_2, s_3) which are the roots of the eigenvalues of Σ. Also, the rotational matrix can be further decomposed into four values (w, x, y, z) called a *quaternion* in the following way [27]:

$$R = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}$$

With the covariance matrix fully decomposed into two smaller components, 3DGS only ever stores these seven values for each 3D Gaussian and treats each component separately in the backpropagation process. A new covariance matrix can be constructed from both components, which is guaranteed to be positive semi-definite and thus geometrically valid since an eigendecomposition is always positive semi-definite [25].

4. Training Procedure for 3D Reconstruction

Training a 3DGS scene starts with a point cloud which contains all positions for the initial 3D Gaussians. Following this, forward and backpropagation passes alternate similar to those in neural networks [29], modifying opacity, position, shape, and color of each 3D Gaussian in the scene. After each error-correcting backward pass, the forward pass renders an image from the same camera pose as the corresponding input image until differences between rendered and input images converge, which takes tens of thousands of iterations. Pivotal for training quality is so-called *Adaptive Density Control*, which periodically steps in after backpropagation, further modifying 3D Gaussians [12].

In this chapter we will closely follow Kerbl et al. [12] and their GitHub repository [11]. The entire training procedure is based on neural/differentiable rendering concepts introduced by Yifan et al. [35] and "Pulsar" by Lassner and Zollhöfer [15] which can be seen as the direct predecessors of the rendering pipeline used for 3DGS. However, there are only very few papers that directly dissect parts of the pipeline math (like [8] or [37] when improving the algorithm) as most papers use 3DGS for various applications only.

This is why much content in this chapter originates from independent research and deductions or from in-depth websites like the formidable "MrNeRF" GitHub repository [21] and others like [5, 18, 39]. These references are valid throughout the chapter.

For more focus, another paper in this seminar deals with neural rendering.

4.1. Point Cloud Initialization

Initialization in 3D reconstruction is a crucial step as the positions of the initial points or primitives decide how well the algorithm can reconstruct a scene.

In order for the 3DGS learning algorithm to work, 3DGS needs initial geometry which is commonly a set of initial 3D Gaussians. This initial set is derived from overlapping input imagery such as photos or frames of a video. If the camera poses of this input imagery are not given directly, which is usually the case, the camera poses have to be derived dynamically from the imagery. A camera pose is designated by a 4×4 *extrinsic matrix* W [17] consisting of the 3×3 rotational matrix W_{rot} which contains the orientation of a camera in 3D space, a 3×1 translation vector t_{trans} denoting the position of the camera in world space, and a padding row to make W square:

$$W = \begin{bmatrix} W_{rot_{11}} & W_{rot_{12}} & W_{rot_{13}} & t_{trans,x} \\ W_{rot_{21}} & W_{rot_{22}} & W_{rot_{23}} & t_{trans,y} \\ W_{rot_{31}} & W_{rot_{32}} & W_{rot_{33}} & t_{trans,z} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This matrix will become important later during forward and backpropagation steps.

The other component of a camera pose is the *intrinsic matrix* which holds parameters of the camera lens like focal lengths.

The de facto standard concept in 3D imaging to derive these two matrices for each input image is called *Structure from Motion* (SfM) [30] offered through a software called *COLMAP*. SfM solves a large geometric optimization problem which finds common key points across all input imagery and uses them to provide highly accurate camera poses and a 3D point cloud of the scene.

3DGS, as originally conceived, uses these points as the origins for the initial 3D Gaussians. They are initialized as perfect spheres with radii deduced from the three nearest neighbors and an opacity of around 0.1. The initial color can be any RGB value.

Consequently, deriving camera poses for initialization first adds significantly to the entire training process, which is why other, faster COLMAP-free methods like *AI-Based Depth Priors* (e.g. [31]) have been proposed. Also, Kerbl et al. [12] mention that randomly initializing all 3D Gaussians yields good quality results despite longer training times.

After initialization, we have valid values for opacity ($o \in (0, 1)$), the quaternions q are all at $(1, 0, 0, 0)^T$ (no rotation), positions μ and colors c are set, and the scales s have been initialized at multiples of $(1, 1, 1)^T$. Finally, the algorithm converts scale and opacity into *latent space* (e.g. [4]) by applying the natural logarithm or the logit function, respectively.

4.2. Forward Pass

At the beginning of each forward pass, the algorithm selects an input image and its camera matrices (see Chapter 3.1) at random. Each forward pass and subsequent backward pass only deal with one and the same input image.

In principle, the forward pass is performing the splatting operation of all 3D Gaussians in the scene to render a new image.

4.2.1. Phases

We decompose the forward pass into six logical phases. For every phase we track all five types of attributes (position, opacity, scale, rotation, and color).

Forward and backward passes both make heavy use of CUDA-based GPUs. In the forward pass, the GPU is running one thread per 3D Gaussian for phases 1 to 4, waits for all threads to finish, performs a global radix sort in phase 5, and switches to running one thread per pixel of the output image for phase 6. For details about the internal workings of modern GPGPUs and related frameworks like Nvidia CUDA see [22].

Phase 1: Preparing Constituent Attributes Scale, rotation, and opacity values are available in so-called *latent space* used for raw, unconstrained values that the optimizer calculated in the previous step of the backward pass. With the color value being retrieved and the position always fully unconstrained, this means the algorithm converts raw scale values to valid ones in *physical space* [4] using the exponential function, normalizes the raw quaternion, and bounds the raw base opacity to $(0, 1)$ using the sigmoid function:

$$s = e^{s_{raw}}, \qquad q = \frac{q_{raw}}{||q_{raw}||}, \qquad o = \frac{1}{1 + e^{-o_{raw}}}$$

This shows why the algorithm had to invert the initial values for s and o after initialization.

Result: 3D Gaussian attributes are available in physical space.

Phase 2: Reconstructing the 3D Covariance Matrix The algorithm rebuilds the 3D covariance matrix according to Chapter 3.1 by creating the scaling matrix S from s and rotational matrix R from q, calculating $\Sigma_{3D} = MM^T$, with $M = RS$.

Result: 3D Covariance Matrix Σ_{3D} is available.

Phase 3: Projection from 3D to 2D We convert 3D Gaussians (ellipsoids) in world space into 2D Gaussians (ellipses) in image space using camera projection. As applying standard pinhole projection [17] is non-linear and would result in non-elliptic shapes for the 2D Gaussian, Zwicker et al. [40] introduced an *affine approximation* of this projection, which is linear and preserves the symmetrical properties of an ellipse. This first-order linear approximation of a non-linear multivariate function is the 2×3 Jacobian matrix J. For that, we move the center of a 3D Gaussian μ_{3D} into view space by applying the rotation and translation of the extrinsic camera matrix (Chapter 4.1), yielding $t = W_{rot} \cdot \mu_{3D} + t_{trans}$, and apply standard pinhole projection to it with focal lengths f_x and f_y:

$$u_x = f_x \frac{t_x}{t_z}, \quad u_y = f_y \frac{t_y}{t_z}, \quad \text{resulting in} \quad J = \begin{bmatrix} \frac{\partial u_x}{\partial t_x} & \frac{\partial u_x}{\partial t_y} & \frac{\partial u_x}{\partial t_z} \\ \frac{\partial u_y}{\partial t_x} & \frac{\partial u_y}{\partial t_y} & \frac{\partial u_y}{\partial t_z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{t_z} & 0 & -f_x \frac{t_x}{t_z^2} \\ 0 & \frac{f_y}{t_z} & -f_y \frac{t_y}{t_z^2} \end{bmatrix}$$

yielding $\Sigma_{2D} = JW_{rot}\Sigma_{3D}W_{rot}^T J^T \in \mathbb{R}^{2\times 2}$. The inner matrix product $W_{rot}\Sigma_{3D}W_{rot}^T$ moves the 3D covariance matrix into view space, and the outer multiplication by the Jacobian J then transforms the 3D Gaussian ellipsoid into a 2D Gaussian ellipse. The center of the 2D Gaussian is $\mu_{2D} = (u_x, u_y)^T \in \mathbb{R}^2$. Color and opacity are not affected in this phase. In fact, the dimension-agnostic formula for opacity distribution (see Chapter 3) is also valid in two dimensions: $\alpha(p) = o \cdot \exp\left(-\frac{1}{2}\Delta^T \Sigma_{2D}^{-1} \Delta\right), \Delta = p - \mu_{2D}$.

Consequently, the algorithm precalculates Σ_{2D}^{-1}, the inverse of the 2D covariance matrix, as only three values A, B, and C in order to avoid frequent recalculation in the last phase:

$$\Sigma_{2D} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}, \quad \Sigma_{2D}^{-1} = \frac{1}{D} \begin{bmatrix} \sigma_{yy} & -\sigma_{xy} \\ -\sigma_{xy} & \sigma_{xx} \end{bmatrix} = \begin{bmatrix} A & B \\ B & C \end{bmatrix}, \quad \text{D the determinant.}$$

Result: Each Gaussian is defined in 3D and 2D.

Phase 4: Preparing Radix Sort First, the final output image will be rendered in tiles of 16×16 pixels. For this, we need to know how many tiles a 2D Gaussian covers, which the algorithm achieves by calculating a 2D bounding box for a 2D Gaussian.

Second, in order to perform splatting in the final phase, we need to know the distance of the center of the *3D* Gaussian from the camera. For the distance, the algorithm does not use the Euclidean/radial distance but *planar Z-depth*, the orthogonal distance from the view plane, which is the camera sensor. Z-depth is the value t_z already available. Discrepancies where two 3D Gaussians a and b have different shapes, $\mu_z^a < \mu_z^b$ but the shape of b is closer to the camera than the shape of a, smooth out over time.

For each tile the bounding box of a 2D Gaussian touches, the algorithm stores a 64-bit entry in VRAM, which consists of the tile ID in the higher 32 bits and the Z-depth in the lower 32 bits (and a pointer to the Gaussian data structure). This de facto is duplication.

Result: An unordered list of 2D Gaussian tile and depth information in VRAM.

Phase 5: Global Radix Sort The GPU waits until all threads have finished calculating their per-Gaussian logic. Radix Sort [34] is a non-comparative sorting algorithm, which runs highly optimized on Nvidia GPUs using a library called *CUB* [1]. For the actual splatting, we need to have a sorted list of all 2D Gaussians per tile. Running Radix Sort once on the prepared unordered list is extremely fast (around 1ms per 10 million entries) and directly sorts all 2D Gaussians by tile and then by depth.

Result: A list of 2D Gaussians sorted by depth (closest first) for every 16×16 tile.

Phase 6: Tile-Based Rasterization We are now in 2D image space. Using the sorted list of Gaussian information we implicitly performed frustum culling with each view frustum having a tile of 16×16 pixels as the view plane. The algorithm now splats all 2D Gaussians in a tile from closest to farthest onto the view plane, adding to the pixel colors.

As the sorted list is per-tile, some 2D Gaussians will not be considered for certain pixels. The massive speed-ups originating from perfectly mapping the 256 pixels to threads in a GPU thread block [12] far outweigh this slight redundancy. This means each thread in the thread block calculates the color of one pixel.

The process of calculating the color of a pixel from a sorted list of colored and transparent objects is called *Alpha Compositing*, is well-known [23, 19], and is also used by NeRFs [20]. The color C_p of a pixel according to alpha compositing is

$$C_p = \sum_{i=1}^{N} c_i \alpha_i T_i, \quad \text{with} \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) \quad \text{the transmittance,}$$

c_i the color of the 2D Gaussian, α_i the opacity of the 2D Gaussian, and N the length of the sorted per-tile list. $\alpha_i = o \cdot \exp(E)$, with $E = -\frac{1}{2}(A \cdot dx^2 + 2B \cdot dx \cdot dy + C \cdot dy^2)$, A, B, and C the 2D covariance matrix values stored after projection in phase 3, and (dx, dy) the distance from the pixel center to the 2D Gaussian center point, simplifying matrix multiplication significantly. The transmittance is the opacity left over at 2D Gaussian i after all 2D Gaussians before i have already been aggregated in the pixel color.

This means each GPU thread runs a loop over all Gaussians in the sorted list from closest to farthest, splatting each 2D Gaussian against the view plane until the transmittance has reached almost 0. This early-stopping shortcut as well as not having to consider empty samples like NeRF along its ray marching path, yields another massive speed-up.

Result: The final rendered image. Pass is complete.

4.3. Loss Function

Kerbl et al. [12] use a weighted combination of regular $\mathcal{L}_1$ loss and *D-SSIM* (a structural similarity index [32]): $L = (1 - \lambda) \cdot \mathcal{L}_1 + \lambda \cdot \mathcal{L}_{\text{D-SSIM}}$, with $\lambda = 0.2$. While $\mathcal{L}_1$ calculates the absolute error as the mean distance between color values of corresponding pixels, D-SSIM, among other aspects, looks at the features and contrast surrounding these pixels.

4.4. Backward Pass

During backpropagation, the loss from comparing the rendered image resulting from the previous forward pass with the original input image is passed backward from the image through to 2D Gaussians and finally the 3D Gaussians via gradient descent in principle identical to neural networks [29]. If the loss is small enough or if the algorithm has reached a maximum number of iterations (usually around 30,000 [12]), backward pass and training algorithm terminate, providing the final reconstructed 3D scene.

4.4.1. Phases

We decompose the backward pass into five logical phases. On the GPU, phases 1 to 3 use the known 16×16 pixel tiles while in phases 4 and 5 threads are running per-Gaussian.

Also, the algorithm still has access to the per-tile lists of 2D Gaussians sorted by depth as well as the computational graph containing intermediate results from the forward pass.

Phase 1: Loss to Pixel Color Correction The derivative of $\mathcal{L}_1 = \frac{1}{N} \sum_{\text{p}=1}^{N} |C_p - O_p|$ at a specific pixel p is $\frac{\partial \mathcal{L}_1}{\partial C_p} = \frac{1}{N} \cdot \text{sign}(C_p - O_p)$ with C the rendered color, O the original color.

$$\text{With } \frac{\partial \mathcal{L}_{\text{D-SSIM}}}{\partial C_p} \text{ this becomes } \frac{\partial L}{\partial C_p} = \frac{1-\lambda}{N} \begin{bmatrix} \text{sign}(R_{C,p} - R_{O,p}) \\ \text{sign}(G_{C,p} - G_{O,p}) \\ \text{sign}(B_{C,p} - B_{O,p}) \end{bmatrix} + \lambda \begin{bmatrix} \frac{\partial \mathcal{L}_{\text{D-SSIM}}}{\partial R_p} \\ \frac{\partial \mathcal{L}_{\text{D-SSIM}}}{\partial G_p} \\ \frac{\partial \mathcal{L}_{\text{D-SSIM}}}{\partial B_p} \end{bmatrix} \in \mathbb{R}^3.$$

While $\mathcal{L}_1$ provides a direction for color alignment, D-SSIM acts as a structural regularizer.

Result: The gradients hold the required color correction per pixel. A positive color component gradient indicates an overestimation, vice versa for negative values, and 0 requires no change. The magnitude dictates the degree of adjustment required.

Phase 2: Pixel Color Correction to 2D Splat In this phase we need to work alpha compositing backwards ($C_p = \sum_{i=1}^{N} c_i \alpha_i T_i$, with $T_i = \prod_{j=1}^{i-1}(1-\alpha_j)$, see phase 6 of the forward pass), meaning the $\frac{\partial L}{\partial C_p}$ gradient flows into the *evaluated* opacity α_i at pixel p and color c_i for each 2D Gaussian in the sorted list for this tile from closest to farthest.

Using the chain rule, for color this is $\frac{\partial L}{\partial c_i} = \frac{\partial L}{\partial C_p} \cdot \frac{\partial C_p}{\partial c_i} = \frac{\partial L}{\partial C_p} \cdot (\alpha_i T_i) \in \mathbb{R}^3$

and for evaluated opacity at pixel p $\frac{\partial L}{\partial \alpha_i} = \frac{\partial L}{\partial C_p} \cdot \frac{\partial C_p}{\partial \alpha_i} = \frac{\partial L}{\partial C_p} \cdot T_i(c_i - C_{behind}) \in \mathbb{R}$,

with C_{behind} the accumulated color of everything behind 2D Gaussian i. Conceptually, this gradient increases evaluated opacity α_i if the 2D Gaussian's color c_i reduces the pixel error more effectively than the accumulated color of the geometry behind it (C_{behind}). On the other hand, if the geometry behind it is more accurate, the gradient decreases α_i to let the background geometry show through.

If the 2D Gaussian spans multiple pixels, the GPU adds all gradients of a type atomically into an aggregated gradient: e.g. Total $\frac{\partial L}{\partial \alpha_i} = \sum_{p \in \text{Pixels}}(\frac{\partial L}{\partial \alpha_i}$ for pixel $p)$. This applies to phase 3 as well.

Result: Final gradients for color c, intermediate gradients for evaluated opacity α.

Phase 3: 2D Splat to 2D Geometry The gradient for *evaluated* opacity α_i now flows into the physical properties of a 2D Gaussian: the actual base opacity o, position μ_{2D}, and 2D covariance matrix Σ_{2D} adjusting 2D geometry by moving and changing shape. For this, we recall the opacity distribution $\alpha_i(p) = o \cdot \exp(-\frac{1}{2}\Delta p^T \Sigma_{2D}^{-1} \Delta p)$ with $\Delta = p - \mu_{2D}$ or simplified with E denoting the exponent: $\alpha_i(p) = o \cdot e^E$.

For the base opacity, the derivative is simple: $\frac{\partial L}{\partial o} = \frac{\partial L}{\partial \alpha_i} \cdot \frac{\partial \alpha_i}{\partial o} = \frac{\partial L}{\partial \alpha_i} \cdot e^E \in \mathbb{R}$.

e^E represents how far the pixel is away from the center of the 2D Gaussian. If the pixel is close to the center, the algorithm increases the base opacity gradient.

For μ_{2D} we want to know how the opacity at pixel p changes when we move it:

Let $v = \Sigma_{2D}^{-1}\Delta p$: $\frac{\partial \alpha_i}{\partial \mu_{2D}} = o \cdot e^E \cdot \frac{\partial E}{\partial \mu_{2D}} = \alpha_i(p) \cdot v$, $\frac{\partial L}{\partial \mu_{2D}} = \frac{\partial L}{\partial \alpha_i} \cdot \frac{\partial \alpha_i}{\partial \mu_{2D}} = \frac{\partial L}{\partial \alpha_i} \cdot \alpha_i(p) \cdot v \in \mathbb{R}^2$

Vector v is pointing directly from the 2D Gaussian's center to the pixel. If the evaluated opacity gradient is positive, the center is effectively pulled towards the pixel.

For Σ_{2D} we want to know how the opacity at pixel p changes when the shape changes. Appendix B.1 lists the derivation of $\frac{\partial \alpha_i}{\partial \Sigma_{2D}}$ which we provide here directly:

$$\frac{\partial L}{\partial \Sigma_{2D}} = \frac{\partial L}{\partial \alpha_i} \cdot \frac{\partial \alpha_i}{\partial \Sigma_{2D}} = \frac{\partial L}{\partial \alpha_i} \cdot \frac{1}{2}\alpha_i(p) \cdot \left(v \cdot v^T\right) \in \mathbb{R}^{2\times 2}$$

Again we have the vector v multiplied by its own transpose which results in a matrix gradient making the 2D Gaussian stretch itself towards the pixel if the evaluated opacity gradient is positive, or else contract.

Result: Intermediate gradients for base opacity o, 2D attributes Σ_{2D} and μ_{2D}.

Phase 4: 2D Geometry to 3D Geometry The GPU model changes to running per-Gaussian. In this phase, the algorithm *unprojects* the 2D Gaussian to the original 3D Gaussian. We recall the camera projection $\Sigma_{2D} = U\Sigma_{3D}U^T$ with $U = JW_{rot}$, J and W_{rot} as per the forward pass, $t \in \mathbb{R}^3$ the view space position of the 3D Gaussian, $\frac{\partial \mu_{2D}}{\partial t} = J$ and $\frac{\partial t}{\partial \mu_{3D}} = W_{rot}$. First we unproject from image space to view space using the multi-variable chain rule, then in the same manner from view space to world space, first for μ_{2D} to μ_{3D}:

$$\text{View space:} \quad \frac{\partial L}{\partial t} = \left(\frac{\partial \mu_{2D}}{\partial t}\right)^T \frac{\partial L}{\partial \mu_{2D}} = J^T \frac{\partial L}{\partial \mu_{2D}} \in \mathbb{R}^3$$

$$\text{World space:} \quad \frac{\partial L}{\partial \mu_{3D}} = \left(\frac{\partial t}{\partial \mu_{3D}}\right)^T \frac{\partial L}{\partial t} = W_{rot}^T \frac{\partial L}{\partial t} = W_{rot}^T J^T \frac{\partial L}{\partial \mu_{2D}} = U^T \left(\frac{\partial L}{\partial \mu_{2D}}\right) \in \mathbb{R}^3$$

Appendix B.2 provides the complete calculation for Σ_{2D} to Σ_{3D}.

$$\text{In short, the final gradient is very clean:} \quad \frac{\partial L}{\partial \Sigma_{3D}} = U^T \left(\frac{\partial L}{\partial \Sigma_{2D}}\right) U \in \mathbb{R}^{3\times 3}$$

The unprojections seem intuitive, using the transposes of the forward projection matrices.

Result: Final gradients for 3D Gaussian center μ_{3D}, intermediate gradients for Σ_{3D}.

Phase 5: 3D Geometry to 3D Gaussian Attributes In the last phase, the algorithm flows the $\frac{\partial L}{\partial \Sigma_{3D}}$ gradient through the 3D covariance matrix assembly into the raw constituent attributes s_{raw} for the scaling matrix S and quaternion q_{raw} for the rotational matrix R. Appendix B.3 provides the exact math behind, we list the major steps here. We recall $\Sigma_{3D} = RSS^TR^T = MM^T$ with $M = RS$. First, we calculate $\frac{\partial L}{\partial M} = 2(\frac{\partial L}{\partial \Sigma_{3D}})M \in \mathbb{R}^{3\times 3}$ and split this into $\frac{\partial L}{\partial S} = R^T(\frac{\partial L}{\partial M})$ for scale and $\frac{\partial L}{\partial R} = (\frac{\partial L}{\partial M})S$ for rotation. By extracting the diagonal from $\frac{\partial L}{\partial S}$ we get $\frac{\partial L}{\partial s} \in \mathbb{R}^{+3}$. For the rotation we calculate the partial derivatives of R (Chapter 3.1) of each of the four components and aggregate those results into $\frac{\partial L}{\partial q} \in \mathbb{R}^4$.

Finally, we have to backpropagate through activation function exp for s_{raw} ($e^{s_{raw}} = s$), the normalization function for q_{raw} as well as the sigmoid function for base opacity o_{raw}:

$$\frac{\partial L}{\partial s_{raw}} = \frac{\partial L}{\partial s} \cdot s \quad \text{and} \quad \frac{\partial L}{\partial q_{raw}} = J_{||}^T \frac{\partial L}{\partial q} \quad \text{with} \quad J_{||} = \frac{I - qq^T}{||q_{raw}||} \quad \text{and} \quad \frac{\partial L}{\partial o_{raw}} = \frac{\partial L}{\partial o} \cdot o(1-o)$$

Result: Final gradients for scale s, rotation q and base opacity o. Pass is complete.

4.4.2. Adaptive Density Control (ADC)

In order to counter over-reconstruction (a 3D Gaussian covering too much geometry) and under-reconstruction (too few 3D Gaussians covering geometry), Kerbl et al. [12] introduced this regulating mechanism, which steps in every 100 iterations after phase 5 of the backward pass. For the former case the 3D Gaussian in question is split into two smaller Gaussians, reduced in size by a factor of 1.6 and positioned slightly apart within the space of the old Gaussian. For the latter case the 3D Gaussian is cloned and the clone's position is moved towards the positional gradient (see Figure 2 on page 13).

New 3D Gaussians simply enter the subsequent forward pass and get adjusted in the pipeline like all other Gaussians.

The authors achieve this by accumulating the 2D positional gradient $\left\| \frac{\partial L}{\partial \mu_{2D}} \right\|$. If the sum has reached a certain threshold when ADC is running, meaning the Gaussian experienced a series of high positional gradients, the algorithm checks if it should split or clone.

Furthermore, there is a pruning step. 3D Gaussians with too low opacity or unbound scale are deleted. In addition to that, every 3,000 iterations, the base opacity is reset to almost 0 for all 3D Gaussians. This causes important ones to regain opacity quickly, while unimportant ones fade out completely and are deleted.

4.4.3. Optimization Step

Where regular gradient descent would simply calculate `new_value = old_value - (learning_rate * gradient)` [29], 3DGS uses the *Adam* optimizer (Adaptive Moment Estimation) [13], which keeps a history t on every parameter p and calculates momentum $m_{t,p}$ and variance $v_{t,p}$, weighing the newest gradient against the ones before, e.g. $m_{t,p} = \beta_p m_{t-1,p} + (1 - \beta_p) \cdot new\_gradient$, β_p the weight. With per-parameter learning rate l_p, $0 < l_p \ll 1$, the new value makes a tiny step: $l_p \cdot \frac{m_{t,p}}{\sqrt{v_{t,p}}+\epsilon}$. Smoothing out over momentum and variance of the past iterations, Adam ensures all 3D Gaussians gently vibrate and morph into their correct geometric shapes, positions, colors, and opacities.

Closing the loop to the next forward pass, all new, raw values reside in latent space.

5. Novel-View Synthesis

Once the scene has been trained as per the previous chapter, the algorithm performs phase 2 of the forward pass once to calculate the 3D covariance matrix for all 3D Gaussians. From then on, novel-view synthesis is feed-forward only, repeating forward pass phases 3 to 6 for every novel view. There is no longer any need for conversions from latent to physical space with the backward pass, Adaptive Density Control, and the Adam optimizer now disabled [11]. We can use the camera matrices of any camera position in the scene to render a new image.

With the raw speed of the tile-based rasterizer, novel-view synthesis in 3DGS quickly reaches real-time levels [12]. Under certain conditions, even 1000+ FPS are possible [38].

6. Conclusion

In a true academic effort, Kerbl et al. combined previously existing research from different fields and proven technologies, linking them with their own approaches, to create a powerful 3D reconstruction pipeline using 3D Gaussian Splatting and appropriate hardware.

This seminar paper aims to have provided a deeper understanding of the concepts of 3D Gaussian Splatting and its fundamental details during initialization, forward pass, and backpropagation within the uncommon setting of a non-neural architecture.

For an audience in Data Science and Computer Science at AIG, this focus sheds light on the inner workings and mathematical connections, which, aggregated as such, were previously unavailable or scattered across several resources.

A. Figures

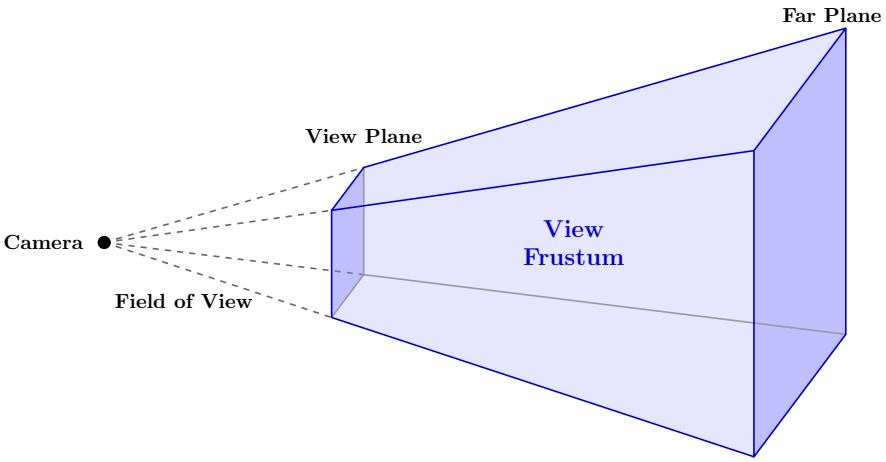

Figure 1: A View Frustum with the frustum in blue. Only Elements inside the frustum between view and far planes are considered.

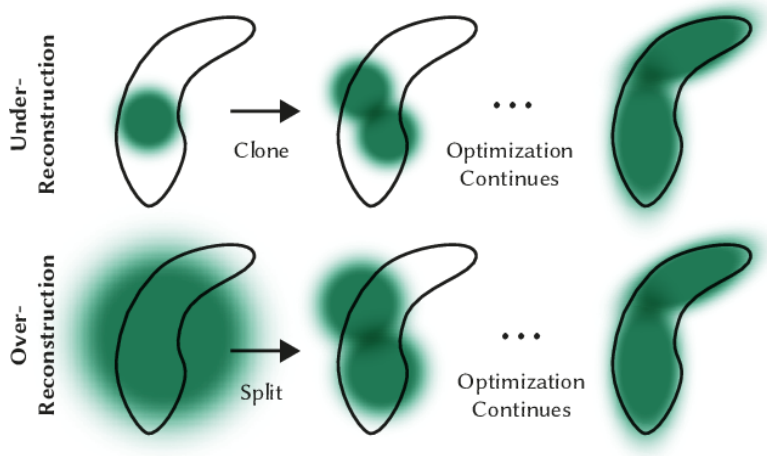

Figure 2: Adaptive Density Control situations for over- and under-reconstruction portraying cloning and splitting (from [12])

B. Complete Backpropagation Calculations

B.1. Phase 3

B.2. Phase 4

B.3. Phase 5

References

[1] Andy Adinets and Duane Merrill. Onesweep: A Faster Least Significant Digit Radix Sort for GPUs, 2022. arXiv preprint arXiv:2206.01784 - Nvidia never published.

[2] Milena T. Bagdasarian, Paul Knoll, Yi-Hsin Li, Florian Barthel, Anna Hilsmann, Peter Eisert, and Wieland Morgenstern. 3DGS.zip: A survey on 3D Gaussian Splatting Compression Methods. *Computer Graphics Forum*, 44(2):e70078, May 2025.

[3] Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5835–5844, 2021.

[4] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation Learning: A Review and New Perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, August 2013.

[5] Tarek Bouamer. A Comprehensive Study for Gaussian Splatting. `https://tarekbouamer.github.io/posts/gaussian-splatting/`, May 2025. Viewed on August 4th, 2026.

[6] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance Fields without Neural Networks. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5491–5500, June 2022.

[7] Shuting He, Ping Ji, Yitong Yang, Changshuo Wang, Jiayi Ji, Yinglin Wang, and Henghui Ding. A Survey on 3D Gaussian Splatting Applications: Segmentation, Editing, and Generation. *IEEE transactions on pattern analysis and machine intelligence*, PP, 2025.

[8] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. In *ACM SIGGRAPH 2024 Conference Papers*, SIGGRAPH '24, pages 1–11, New York, NY, USA, July 2024. Association for Computing Machinery.

[9] Arthur Hubert, Gamal Elghazaly, and Raphaël Frank. Editing Implicit and Explicit Representations of Radiance Fields: A Survey. *Machine Vision and Applications*, 36(6):119, September 2025.

[10] INRIA. Project Website for Kerbl et al.: 3D Gaussian Splatting for Real-Time Radiance Field Rendering. `https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting`. Viewed on August 4th, 2026.

[11] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. GitHub Repository: gaussian-splatting. `https://github.com/graphdeco-inria/gaussian-splatting`. Viewed on August 4th, 2026.

[12] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4):1–14, August 2023.

[13] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015.

[14] Georgios Kopanas, Julien Philip, Thomas Leimkühler, and George Drettakis. Point-Based Neural Rendering with Per-View Optimization. *Computer Graphics Forum*, 40(4):29–43, July 2021.

[15] Christoph Lassner and Michael Zollhöfer. Pulsar: Efficient Sphere-based Neural Rendering. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1440–1449, June 2021.

[16] Shuai Liu, Mengmeng Yang, Tingyan Xing, and Ran Yang. A Survey of 3D Reconstruction: The Evolution from Multi-View Geometry to NeRF and 3DGS. *Sensors*, 25(18):5748, January 2025.

[17] Steve Marschner and Peter Shirley. *Fundamentals of Computer Graphics*. A K Peters/CRC Press, New York, 5th edition, September 2021.

[18] Marcin Matuszczyk. Rendering in 3D Gaussian Splatting. `https://www.sctheblog.com/blog/gaussian-splatting/`, June 2024. Viewed on August 4th, 2026.

[19] Nelson Max. Optical Models for Direct Volume Rendering. *IEEE Transactions on Visualization and Computer Graphics*, 1(2):99–108, June 1995.

[20] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: representing scenes as neural radiance fields for view synthesis. *Commun. ACM*, 65(1):99–106, December 2021.

[21] MrNeRF. GitHub Repository: Awesome 3D Gaussian Splatting. `https://github.com/MrNeRF/awesome-3D-gaussian-splatting`. Viewed on August 4th, 2026.

[22] Lena Oden. Advanced Parallel Computing. University course, Chair of Technical Computer Science, Fernuniversität Hagen, 2026.

[23] Thomas Porter and Tom Duff. Compositing Digital Images. *ACM SIGGRAPH Computer Graphics*, 18(3):253–259, January 1984.

[24] researchgate.net. ResearchGate: 3D Gaussian Splatting for Real-Time Radiance Field Rendering. `https://www.researchgate.net/publication/372667406_3D_Gaussian_Splatting_for_Real-Time_Radiance_Field_Rendering`. Viewed on August 4th, 2026.

[25] Sebastian Riedel. Mathematische Grundlagen für Data Science. University script, Applied Stochastics Group, Fernuniversität Hagen, 2026.

[26] semanticscholar.org. SemanticScholar: 3D Gaussian Splatting for Real-Time Radiance Field Rendering. `https://www.semanticscholar.org/paper/3D-Gaussian-Splatting-for-Real-Time-Radiance-Field-Kerbl-Kopanas/2cc1d857e86d5152ba7fe6a8355c2a0150cc280a`. Viewed on August 4th, 2026.

[27] Ken Shoemake. Animating Rotation with Quaternion Curves. *Proceedings of the 12th annual conference on Computer graphics and interactive techniques*, 1985.

[28] Ayush Tewari, Ohad Fried, Justus Thies, Vincent Sitzmann, Stephen Lombardi, Kalyan Sunkavalli, Ricardo Martin-Brualla, Tomas Simon, Jason Saragih, Matthias Nießner, Rohit Pandey, Sean Fanello, Gordon Wetzstein, Jun-Yan Zhu, Christian Theobalt, Maneesh Agrawala, Eli Shechtman, Dan B. Goldman, and Michael Zollhöfer. State of the Art on Neural Rendering. *Eurographics*, 39(2), April 2020.

[29] Matthias Thimm. Maschinelles Lernen. University script, Artificial Intelligence Group, Fernuniversität Hagen, 2026.

[30] S. Ullman. The Interpretation of Structure from Motion. *Proceedings of the Royal Society of London. B. Biological Sciences*, 203(1153):405–426, January 1979.

[31] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. DUSt3R: Geometric 3D Vision Made Easy. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20697–20709, June 2024.

[32] Zhou Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, April 2004.

[33] Lee Alan Westover. *Splatting: A Parallel, Feed-Forward Volume Rendering Algorithm.* PhD thesis, The University of North Carolina at Chapel Hill, 1991.

[34] wikipedia.org. Radix Sort. `https://en.wikipedia.org/wiki/Radix_sort`. Viewed on August 4th, 2026.

[35] Wang Yifan, Felice Serena, Shihao Wu, Cengiz Öztireli, and Olga Sorkine-Hornung. Differentiable Surface Splatting for Point-Based Geometry Processing. *ACM Transactions on Graphics (TOG)*, 38(6):230:1–230:14, November 2019.

[36] Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, and Angjoo Kanazawa. PlenOctrees for Real-time Rendering of Neural Radiance Fields. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, page 5732–5741, 2021.

[37] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-Splatting: Alias-Free 3D Gaussian Splatting. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19447–19456, June 2024.

[38] Yuheng Yuan, Qiuhong Shen, Xingyi Yang, and Xinchao Wang. 1000+ FPS 4D Gaussian Splatting for Dynamic Scene Rendering. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, pages 74665–74683. Curran Associates, Inc., 2025.

[39] Kate Yurkova. A Comprehensive Overview of Gaussian Splatting. `https://towardsdatascience.com/a-comprehensive-overview-of-gaussian-splatting-e7d570081362/`, December 2023. Viewed on August 4th, 2026.

[40] Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross. EWA Volume Splatting. In *Proceedings Visualization, 2001*, VIS '01, pages 29–36, San Diego, CA, USA, 2001. IEEE.

[41] Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross. Surface Splatting. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '01, pages 371–378, New York, NY, USA, 2001. Association for Computing Machinery.

Statement

I declare that I have written the seminar paper independently and without unauthorized use of third parties. I have only used the indicated resources and I have clearly marked the passages taken verbatim or in the sense of these resources as such. The assurance of independent work also applies to any drawings, sketches or graphical representations. The work has not previously been submitted in the same or similar form to the same or another examination authority and has not been published. By submitting the electronic version of the final version of the paper, I acknowledge that it will be checked by a plagiarism detection service to check for plagiarism and that it will be stored exclusively for examination purposes.

. . . . . .

(Place, Date) (Signature)